

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/I	Regulation	R23
Date and Day of Assignment	Week 2 - Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques		Week2 - Wednesday

	Lab Objectives - To explore and apply different levels of prompt examples in AI-assisted code generation - To understand how zero-shot, one-shot, and few-shot prompting affect AI output quality - To evaluate the impact of context richness and example quantity on AI performance - To build awareness of prompt strategy effectiveness for different problem types Lab Outcomes (LOs) - After completing this lab, students will be able to: - Use zero-shot prompting to instruct AI with minimal context - Use one-shot prompting with a single example to guide AI code generation - Apply few-shot prompting using multiple examples to improve AI responses - Compare AI outputs across different prompting strategies	
	Task 1: Zero-Shot Prompting – Leap Year Check Scenario Zero-shot prompting involves giving instructions without providing examples. Task Description - Use zero-shot prompting to instruct an AI tool to generate a Python function that: - Accepts a year as input - Checks whether the given year is a leap year - Returns an appropriate result Note: No input-output examples should be provided in the prompt. Expected Output - AI-generated leap year checking function - Correct logical conditions - Sample input and output - Screenshot of AI-generated response (if required)	

```

Year: 2000; / LEAP YEAR
= 2000 is a leap year (has 366 days)

Year: 2004; / LEAP YEAR
= 2004 is a leap year (has 366 days)

Year: 2024; / LEAP YEAR
= 2024 is a leap year (has 366 days)

Year: 2025; X NOT LEAP YEAR
= 2025 is not a leap year (has 365 days)

Year: 1900; X NOT LEAP YEAR
= 1900 is not a leap year (has 365 days)

Year: 2100; X NOT LEAP YEAR
= 2100 is not a leap year (has 365 days)

Year: 2400; / LEAP YEAR
= 2400 is a leap year (has 366 days)

Year: 1906; / LEAP YEAR
= 1906 is a leap year (has 366 days)

```

LOGICAL CONDITIONS APPLIED:

```

1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

C)Answer:
1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

1. Divisible by 400? = Leap Year
2. Divisible by 4 AND NOT divisible by 100? = Leap Year
3. Otherwise = Not a Leap Year

```

	Description: What it is: Zero-shot prompting leap year checker Input: Accepts a year as input Logic: Divisible by 400 OR (divisible by 4 AND NOT by 100) Output: Returns boolean result with descriptive message Examples: Demonstrates with sample test years	
	Task 2: One-Shot Prompting – Centimeters to Inches Conversion Scenario One-shot prompting guides AI using a single example. Task Description Use one-shot prompting by providing one input-output example to generate a Python function that: Converts centimeters to inches Uses the correct mathematical formula Example provided in prompt: Input: 10 cm → Output: 3.94 inches Expected Output Python function with correct conversion logic Accurate calculation Sample test cases and outputs	

```

C:\Users\prade> prade > OneDrive > Pradeep > cmToInch.py > ...
1 # One-Shot Prompting: Centimeters to Inches Conversion
2 # Example provided: 10 cm → 3.94 inches
3
4 def cm_to_inches(centimeters):
5     """
6     Converts centimeters to inches using the conversion formula.
7
8     Formula: inches = centimeters / 2.54
9
10    Args:
11        centimeters (float): The value in centimeters to convert
12
13    Returns:
14        float: The equivalent value in inches
15    """
16    inches = centimeters / 2.54
17    return round(inches, 2)
18
19
20 # Test Cases
21 print("Centimeters to Inches Conversion")
22 print("-" * 40)
23
24 # Test case 1: Example provided (10 cm → 3.94 inches)
25 test_value_1 = 10
26 result_1 = cm_to_inches(test_value_1)
27 print(f"Input: {test_value_1} cm → Output: {result_1} inches")
28
29 # Test case 2: 5 cm
30 test_value_2 = 5
31 result_2 = cm_to_inches(test_value_2)
32 print(f"Input: {test_value_2} cm → Output: {result_2} inches")
33
34 # Test case 3: 100 cm (1 meter)
35 test_value_3 = 100
36 result_3 = cm_to_inches(test_value_3)
37 print(f"Input: {test_value_3} cm → Output: {result_3} inches")
38
39 # Test case 4: 2.54 cm (exactly 1 inch)
40 test_value_4 = 2.54
41 result_4 = cm_to_inches(test_value_4)
42 print(f"Input: {test_value_4} cm → Output: {result_4} inches")
43
44 # Test case 5: 30 cm
45 test_value_5 = 30
46 result_5 = cm_to_inches(test_value_5)
47 print(f"Input: {test_value_5} cm → Output: {result_5} inches")
48
49 print("-" * 40)
50 print("Conversion complete!")
51

```

Output:

```

C:\Users\prade> C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:/Users/prade/OneDrive/Pradeep/cmToInch.py
Centimeters to Inches Conversion
=====
Input: 10 cm → Output: 3.94 inches
Input: 5 cm → Output: 1.97 inches
Input: 10 cm → Output: 3.94 inches
Input: 5 cm → Output: 1.97 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 5 cm → Output: 1.97 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
=====
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
=====
Conversion complete!
Input: 30 cm → Output: 11.81 inches
=====
Conversion complete!
=====
Conversion complete!
Conversion complete!

```

Description:

The purpose (one-shot prompting demonstration)

How it works (single example guidance)

What it converts (centimeters to inches)

The conversion formula used (1 inch = 2.54 cm)

What's included (multiple test cases for verification)

Task 3: Few-Shot Prompting – Name Formatting Scenario

Few-shot prompting improves accuracy by providing multiple examples.

Task Description

Use few-shot prompting with 2–3 examples to generate a Python function that:

Accepts a full name as input

Formats it as “Last, First”

Example formats:

"John Smith" → "Smith, John"

"Anita Rao" → "Rao, Anita"

Expected Output

Well-structured Python function

Output strictly following example patterns

Correct handling of names

Sample inputs and outputs

```
C:\Users\prade > OneDrive > Pycharm > formatOfName.py > ...
1
2
3 Few-Shot Prompting - Name Formatting
4 This script demonstrates few-shot prompting by providing examples to format names
5 from "First Last" format to "Last, First" format.
6
7
8 # Few-Shot Examples (Prompt)
9
10 examples = [
11     ("input": "John Smith", "output": "Smith, John"),
12     ("input": "Anita Rao", "output": "Rao, Anita"),
13     ("input": "Maria Garcia", "output": "Garcia, Maria"),
14 ]
15
16 def format_name(full_name):
17     """
18     Format a full name from "First Last" to "Last, First" format.
19
20     Args:
21         full_name (str): A full name in "First Last" format
22
23     Returns:
24         str: Formatted name in "Last, First" format
25
26     Examples:
27     >>> format_name("John Smith")
28     Smith, John
29     >>> format_name("Anita Rao")
30     Rao, Anita
31     """
32     # Strip whitespace and split the name
33     parts = full_name.strip().split()
34
35     # Handle edge cases
36     if len(parts) < 2:
37         return full_name # Return as-is if only one name part
38
39     # Separate first and last name (support for middle names/initials)
40     first_name = parts[0]
41     last_name = parts[-1]
42
43     # Format and return
44     return f"{last_name}, {first_name}"
45
46
47 def batch_format_names(names):
48     """
49     Format multiple names at once.
50
51     Args:
52         names (list): List of full names
53
54     Returns:
55         list: List of formatted names
56     """
57     return [format_name(name) for name in names]
```

```
C:\Users\prade > OneDrive > Pycharm > formatOfName.py > ...
58
59 if __name__ == "__main__":
60     print("\n" * 50)
61     print("Few-Shot Prompting - Name Formatting")
62     print("\n" * 50)
63
64     # Display few-shot examples
65     print("\nFew-Shot Examples:")
66     print("\n" * 50)
67     for example in examples:
68         print(f"Input: {example['input']}")
69         print(f"Output: {example['output']}")
70         print()
71
72     # Test the function
73     print("Testing the format_name() function:")
74     print("\n" * 50)
75
76     test_names = [
77         "John Smith",
78         "Anita Rao",
79         "Maria Garcia",
80         "Christopher Lee",
81         "Priya Patel",
82     ]
83
84     for name in test_names:
85         formatted = format_name(name)
86         print(f"({name}) -> {formatted}")
87
88     print("\n" * 50)
89     print("Batch Processing Example:")
90     print("\n" * 50)
91
92     batch_names = ["Alice Johnson", "Bob Williams", "Carol Davis"]
93     batch_results = batch_format_names(batch_names)
94
95     for original, formatted in zip(batch_names, batch_results):
96         print(f"({original}) -> {formatted}")
97
98     print("\n" * 50)
```

Output:

```
C:\Users\pradee>C:\Users\pradee\AppData\Local\Microsoft\WindowsApps\python3.11.exe c:\Users\pradee\OneDrive\Pradeep\formatOfName.py
=====
Few-Shot Prompting - Name Formatting
=====

Few-Shot Examples:
-----
Input: 'John Smith'
Output: 'Smith, John'

Input: 'Anita Rao'
Output: 'Rao, Anita'

Input: 'Maria Garcia'
Output: 'Garcia, Maria'

Testing the format_name() function:
-----
'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Maria Garcia' -> 'Garcia, Maria'
'Christopher Lee' -> 'Lee, Christopher'
'Priya Patel' -> 'Patel, Priya'

=====
Batch Processing Example
-----

Testing the format_name() function:
-----
'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Christopher Lee' -> 'Lee, Christopher'
'Priya Patel' -> 'Patel, Priya'

=====
Batch Processing Example:
-----
'Alice Johnson' -> 'Johnson, Alice'
'Bob Williams' -> 'Williams, Bob'
'Carol Davis' -> 'Davis, Carol'
=====
```

Description:
The few-shot prompting concept
How the script converts name formats
The technical approach used
Batch processing capability
The educational value of the examples

Task 4: Comparative Analysis – Zero-Shot vs Few-Shot Scenario
Different prompt strategies may produce different code

quality.

Task Description

Use zero-shot prompting to generate a function that counts vowels in a string

Use few-shot prompting for the same problem

Compare both outputs based on:

Accuracy

Readability

Logical clarity

Expected Output

Two vowel-counting functions

Comparison table or short reflection paragraph

Conclusion on prompt effectiveness

```

1 # Count vowels in a string using Zero-shot prompting
2
3 # Create a prompt for the Zero-shot function
4 prompt_zero = """
5 Write a function that counts vowels in a string.
6 """
7
8 # Create a prompt for the Few-shot function
9 prompt_few = """
10 Write a function that counts vowels in a string.
11 Here are some examples:
12
13 count_vowels('hello') == 2
14 count_vowels('programming') == 3
15 count_vowels('AEIOUaeiou') == 10
16 count_vowels('Python is awesome!') == 6
17 count_vowels('xyz') == 0
18 """
19
20 # Define the Zero-shot function
21 def count_vowels_zero_shot(text):
22     """Count vowels in a string using Zero-shot prompting"""
23     # Extract the function definition from the prompt
24     function_def = """
25 def count_vowels(text):
26     """Count vowels in a string"""
27     # Your code here
28     return count
29 """
30     # Execute the function
31     exec(function_def)
32     return count_vowels(text)
33
34 # Define the Few-shot function
35 def count_vowels_few_shot(text):
36     """Count vowels in a string using Few-shot prompting"""
37     # Extract the function definition from the prompt
38     function_def = """
39 def count_vowels(text):
40     """Count vowels in a string"""
41     # Your code here
42     return count
43 """
44     # Execute the function
45     exec(function_def)
46     return count_vowels(text)
47
48 # Test the functions
49 test_cases = ['hello', 'programming', 'AEIOUaeiou', 'Python is awesome!', 'xyz']
50
51 for test_case in test_cases:
52     zero_shot_result = count_vowels_zero_shot(test_case)
53     few_shot_result = count_vowels_few_shot(test_case)
54
55     # Print the results
56     print(f"Input: '{test_case}'")
57     print(f"Zero-Shot Result: {zero_shot_result}")
58     print(f"Few-Shot Result: {few_shot_result}")
59     print(f"Results Match: {zero_shot_result == few_shot_result}")
60
61 # Print the total counts
62 zero_shot_total = sum(count_vowels_zero_shot(test_case) for test_case in test_cases)
63 few_shot_total = sum(count_vowels_few_shot(test_case) for test_case in test_cases)
64
65 print(f"Zero-Shot Total: {zero_shot_total}")
66 print(f"Few-Shot Total: {few_shot_total}")
67 print(f"Comparison: {zero_shot_total == few_shot_total}")
68
69 # Print the conclusion
70 print(f"Conclusion: {'Both functions are accurate' if zero_shot_total == few_shot_total else 'Functions differ'}")

```

Output:

```

C:\Users\prado>cd C:\Users\prado\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:\Users\prado\OneDrive\Pradap\count_vowels.py

COMPARATIVE ANALYSIS - ZERO-SHOT VS FEW-SHOT PROMPTING
=====

TEST RESULTS:
=====

Input: 'hello'
Zero-Shot Result: 2
Few-Shot Result: 2 (breakdown: {'a': 0, 'e': 1, 'i': 0, 'o': 1, 'u': 0})
Results Match: True ✓

Input: 'programming'
Zero-Shot Result: 3
Few-Shot Result: 3 (breakdown: {'a': 1, 'e': 0, 'i': 1, 'o': 1, 'u': 0})
Results Match: True ✓

Input: 'AEIOUaeiou'
Zero-Shot Result: 10
Few-Shot Result: 10 (breakdown: {'a': 2, 'e': 2, 'i': 2, 'o': 2, 'u': 2})
Results Match: True ✓

Input: 'Python is awesome!'
Zero-Shot Result: 6
Few-Shot Result: 6 (breakdown: {'a': 1, 'e': 2, 'i': 1, 'o': 2, 'u': 0})
Results Match: True ✓

Input: 'xyz'
Zero-Shot Result: 0
Few-Shot Result: 0 (breakdown: {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0})
Results Match: True ✓

```

Description:

	Zero-shot prompting produces straightforward and easily understandable code. Few-shot prompting results in more concise and optimized solutions. Both approaches are accurate and logically correct. Few-shot improves code quality by learning from examples. Overall, few-shot prompting is more effective for cleaner and professional code, while zero-shot is better for learning basics.	
	Task 5: Few-Shot Prompting – File Handling Scenario File processing requires clear logical understanding. Task Description Use few-shot prompting to generate a Python function that: - Reads a .txt file - Counts the number of lines in the file - Returns the line count Expected Output Working Python file-processing function Correct line count Sample .txt input and output AI-assisted logic explanation	

```

C:\Users\prade> OneDrive\Pradeep> file_handling.py > ...
1 def create_and_read_file():
2
3
4
5     """How the code works:
6     1. Opens or creates a file named 'sample.txt' in write mode
7     2. Writes 6 lines of sample text to the file
8     3. Closes the file
9     4. Opens the file again in read mode
10    5. Counts the total number of lines using a generator expression
11    6. Prints the result in the required format
12
13    Returns:
14        int: The total number of lines in the file
15    """
16    filename = "sample.txt"
17
18    # Sample text to write to the file
19    sample_lines = [
20        "This is the first line of sample text.\n",
21        "Python is a great programming language.\n",
22        "File handling is an important skill in programming.\n",
23        "This program creates and reads a text file.\n",
24        "It counts the number of lines automatically.\n",
25        "This is the sixth line for extra content.\n"
26    ]
27
28    # Create and write to the file
29    try:
30        with open(filename, 'w') as file:
31            file.writelines(sample_lines)
32        print(f"File '{filename}' created successfully.")
33    except IOError:
34        print(f"Error: Unable to create file '{filename}'.")
35        return -1
36
37    # Read and count lines in the file
38    try:
39        with open(filename, 'r') as file:
40            line_count = sum(1 for line in file)
41
42    # Print the result in the required format
43    print(f"The file '{filename}' has {line_count} lines.")
44    return line_count
45
46    except FileNotFoundError:
47        print(f"Error: File '{filename}' not found.")
48        return -1
49    except IOError:
50        print(f"Error: Unable to read file '{filename}'.")
51        return -1
52
53
54 # Main program
55 if __name__ == "__main__":
56     print("-" * 50)
57     print("File Creation and Line Counter Program")
58     print("-" * 50)
59     print()
60
61     create_and_read_file()
62
63     print()
64     print("-" * 50)

```

Output:

Active code page: 65001

```

C:\Users\prade> C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:\Users\prade\OneDrive\Pradeep\file_handling.py
=====
File Creation and Line Counter Program
Active code page: 65001

C:\Users\prade> C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:\Users\prade\OneDrive\Pradeep\file_handling.py
=====
File Creation and Line Counter Program
=====

File 'sample.txt' created successfully.
The file 'sample.txt' has 6 lines.

=====

```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.